



Bank Customer Churn Prediction: Bagging, AdaBoost, XGBoost & Stacking

This notebook applies four **ensemble techniques** to the Bank Customer Churn dataset (`Churn_Modelling.csv`) to predict whether a customer will leave the bank (`Exited` = 1) or stay (`Exited` = 0).

Models applied (each in its own section):

1. **Bagging** (BaggingClassifier with decision trees)
2. **AdaBoost** (Adaptive Boosting)
3. **XGBoost** (Extreme Gradient Boosting)
4. **Stacking** (meta-model over diverse base learners)

Steps covered:

1. Import libraries
2. Load the dataset
3. Explore the data (EDA)
4. Clean and preprocess (drop IDs, encode categoricals)
5. Train/test split & feature scaling
6. Model 1: Bagging
7. Model 2: AdaBoost
8. Model 3: XGBoost
9. Model 4: Stacking
10. Final model comparison

1. Import Libraries

Requires `xgboost` (`pip install xgboost`).

```
In [1]: !pip install xgboost
```

Collecting xgboost

Downloading xgboost-3.2.0-py3-none-win_amd64.whl.metadata (2.1 kB)

Requirement already satisfied: numpy in c:\python311\lib\site-packages (from xgboost) (1.26.4)

Requirement already satisfied: scipy in c:\python311\lib\site-packages (from xgboost) (1.16.3)

Downloading xgboost-3.2.0-py3-none-win_amd64.whl (101.7 MB)

```
----- 0.0/101.7 MB ? eta -:-:--
----- 1.6/101.7 MB 8.3 MB/s eta 0:00:12
----- 3.9/101.7 MB 10.2 MB/s eta 0:00:10
----- 6.3/101.7 MB 10.7 MB/s eta 0:00:09
----- 8.4/101.7 MB 11.1 MB/s eta 0:00:09
----- 9.4/101.7 MB 9.6 MB/s eta 0:00:10
----- 11.0/101.7 MB 8.9 MB/s eta 0:00:11
----- 11.5/101.7 MB 8.2 MB/s eta 0:00:11
----- 13.6/101.7 MB 8.3 MB/s eta 0:00:11
----- 15.5/101.7 MB 8.5 MB/s eta 0:00:11
----- 17.8/101.7 MB 8.7 MB/s eta 0:00:10
----- 20.7/101.7 MB 9.1 MB/s eta 0:00:09
----- 23.1/101.7 MB 9.4 MB/s eta 0:00:09
----- 25.7/101.7 MB 9.6 MB/s eta 0:00:08
----- 28.6/101.7 MB 9.8 MB/s eta 0:00:08
----- 31.2/101.7 MB 10.0 MB/s eta 0:00:08
----- 33.0/101.7 MB 9.9 MB/s eta 0:00:07
----- 34.3/101.7 MB 9.8 MB/s eta 0:00:07
----- 36.7/101.7 MB 9.8 MB/s eta 0:00:07
----- 39.3/101.7 MB 10.0 MB/s eta 0:00:07
----- 43.0/101.7 MB 10.3 MB/s eta 0:00:06
----- 45.9/101.7 MB 10.4 MB/s eta 0:00:06
----- 48.8/101.7 MB 10.6 MB/s eta 0:00:05
----- 51.6/101.7 MB 10.7 MB/s eta 0:00:05
----- 55.1/101.7 MB 11.0 MB/s eta 0:00:05
----- 57.9/101.7 MB 11.1 MB/s eta 0:00:04
----- 60.6/101.7 MB 11.1 MB/s eta 0:00:04
----- 62.1/101.7 MB 11.0 MB/s eta 0:00:04
----- 65.0/101.7 MB 11.1 MB/s eta 0:00:04
----- 67.9/101.7 MB 11.2 MB/s eta 0:00:04
----- 70.3/101.7 MB 11.2 MB/s eta 0:00:03
----- 72.9/101.7 MB 11.2 MB/s eta 0:00:03
----- 75.5/101.7 MB 11.2 MB/s eta 0:00:03
----- 77.9/101.7 MB 11.3 MB/s eta 0:00:03
----- 80.7/101.7 MB 11.3 MB/s eta 0:00:02
----- 82.6/101.7 MB 11.3 MB/s eta 0:00:02
----- 85.2/101.7 MB 11.3 MB/s eta 0:00:02
----- 87.8/101.7 MB 11.3 MB/s eta 0:00:02
----- 89.7/101.7 MB 11.2 MB/s eta 0:00:02
----- 91.8/101.7 MB 11.2 MB/s eta 0:00:01
----- 94.1/101.7 MB 11.2 MB/s eta 0:00:01
----- 96.7/101.7 MB 11.2 MB/s eta 0:00:01
----- 99.4/101.7 MB 11.2 MB/s eta 0:00:01
----- 101.4/101.7 MB 11.3 MB/s eta 0:00:01
----- 101.4/101.7 MB 11.3 MB/s eta 0:00:01
----- 101.7/101.7 MB 10.9 MB/s 0:00:09
```

Installing collected packages: xgboost

Successfully installed xgboost-3.2.0

WARNING: Ignoring invalid distribution ~umpy (c:\Python311\Lib\site-packages)
WARNING: Ignoring invalid distribution ~umpy (c:\Python311\Lib\site-packages)
WARNING: Ignoring invalid distribution ~umpy (c:\Python311\Lib\site-packages)

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import (
    BaggingClassifier, AdaBoostClassifier,
    RandomForestClassifier, StackingClassifier
)
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, roc_curve
)

pd.set_option('display.max_columns', 50)
sns.set_style('whitegrid')
%matplotlib inline
```

2. Load the Dataset

Place `Churn_Modelling.csv` in the same folder as this notebook.

```
In [3]: df = pd.read_csv(r'C:\Users\tusha\Downloads\Churn_Modelling.csv')
print("Shape:", df.shape)
df.head()
```

Shape: (10000, 14)

```
Out[3]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
0	1	15634602	Hargrave	619	France	Female	42	3
1	2	15647311	Hill	608	Spain	Female	41	1
2	3	15619304	Onio	502	France	Female	42	4
3	4	15701354	Boni	699	France	Female	39	4
4	5	15737888	Mitchell	850	Spain	Female	43	3

3. Explore the Data (EDA)

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber              10000 non-null  int64
1   CustomerId             10000 non-null  int64
2   Surname                 10000 non-null  object
3   CreditScore             10000 non-null  int64
4   Geography               10000 non-null  object
5   Gender                  10000 non-null  object
6   Age                     10000 non-null  int64
7   Tenure                  10000 non-null  int64
8   Balance                 10000 non-null  float64
9   NumOfProducts           10000 non-null  int64
10  HasCrCard               10000 non-null  int64
11  IsActiveMember          10000 non-null  int64
12  EstimatedSalary         10000 non-null  float64
13  Exited                  10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
In [5]: # Missing values check
df.isnull().sum()
```

```
Out[5]: RowNumber      0
        CustomerId     0
        Surname         0
        CreditScore     0
        Geography       0
        Gender          0
        Age             0
        Tenure          0
        Balance         0
        NumOfProducts   0
        HasCrCard       0
        IsActiveMember  0
        EstimatedSalary 0
        Exited          0
        dtype: int64
```

```
In [6]: # Target distribution – note the class imbalance (~20% churn)
print(df['Exited'].value_counts())
print(f"\nChurn rate: {df['Exited'].mean():.2%}")

plt.figure(figsize=(5,4))
sns.countplot(x='Exited', data=df, palette=['#4C72B0', '#DD8452'])
plt.title('Target Distribution (0 = Stayed, 1 = Churned)')
plt.show()
```

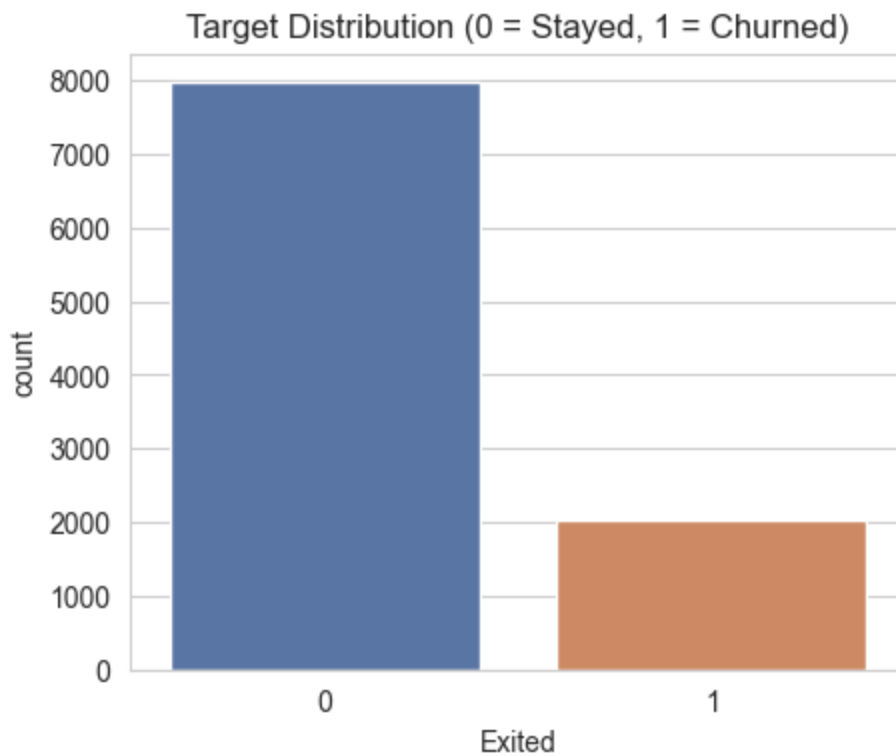
```
Exited
0      7963
1       2037
Name: count, dtype: int64
```

Churn rate: 20.37%

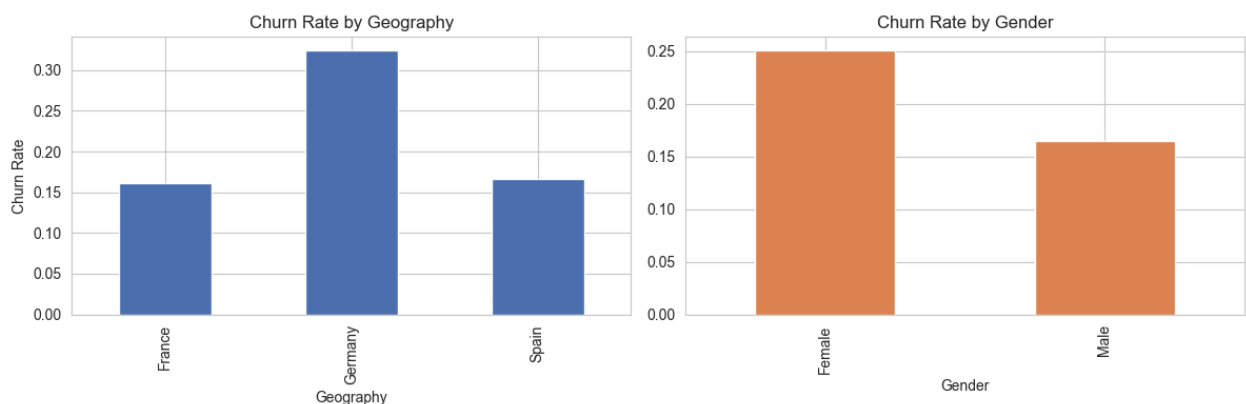
C:\Users\tusha\AppData\Local\Temp\ipykernel_13088\3106162532.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

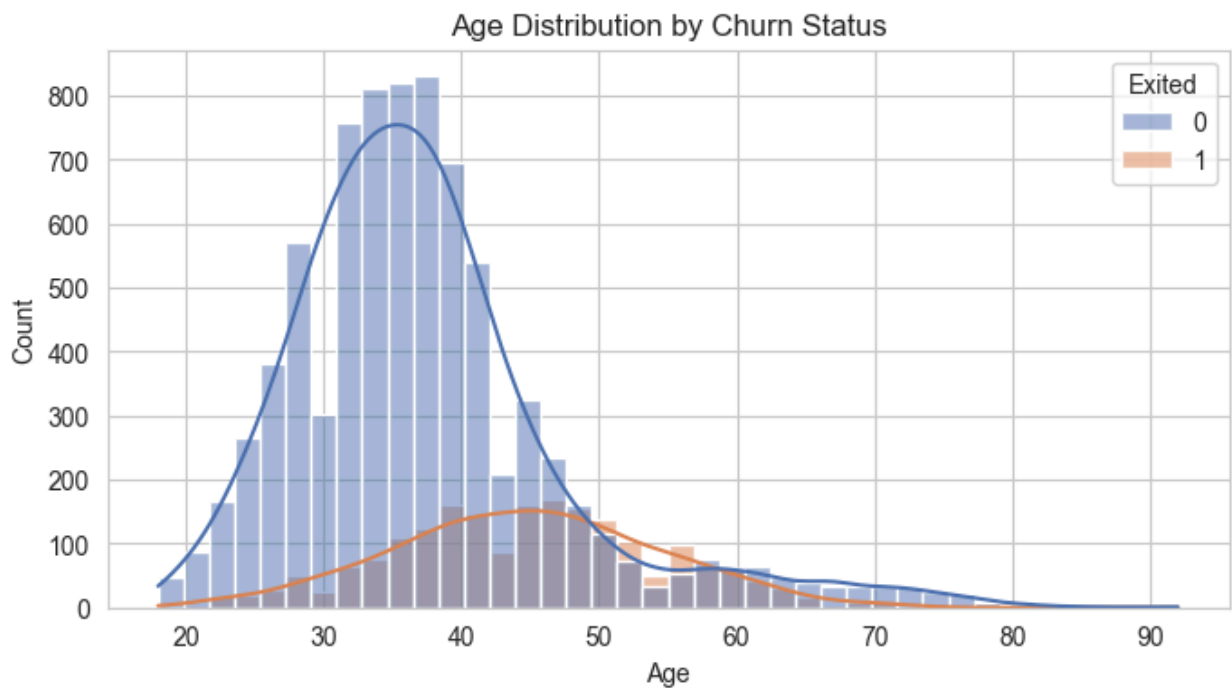
```
sns.countplot(x='Exited', data=df, palette=['#4C72B0', '#DD8452'])
```



```
In [7]: # Churn rate by Geography and Gender
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df.groupby('Geography')['Exited'].mean().plot(kind='bar', ax=axes[0], color='#4C78A8')
axes[0].set_title('Churn Rate by Geography')
axes[0].set_ylabel('Churn Rate')
df.groupby('Gender')['Exited'].mean().plot(kind='bar', ax=axes[1], color='#DD8033')
axes[1].set_title('Churn Rate by Gender')
plt.tight_layout()
plt.show()
```



```
In [8]: # Age distribution by churn status
plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='Age', hue='Exited', bins=40, kde=True, palette=['#4C78A8', '#DD8033'])
plt.title('Age Distribution by Churn Status')
plt.show()
```



4. Clean and Preprocess

- Drop identifier columns (`RowNumber` , `CustomerId` , `Surname`) — not predictive.
- Encode `Gender` with label encoding (binary) and `Geography` with one-hot encoding.

```
In [9]: # Drop identifier columns
df = df.drop(columns=['RowNumber', 'CustomerId', 'Surname'])

# Encode Gender (binary: Female=0, Male=1)
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
print(dict(zip(le.classes_, le.transform(le.classes_))))

# One-hot encode Geography
df = pd.get_dummies(df, columns=['Geography'], drop_first=True, dtype=int)

print("Shape after preprocessing:", df.shape)
df.head()
```

```
{'Female': 0, 'Male': 1}
Shape after preprocessing: (10000, 12)
```

Out[9]:

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	Is...
0	619	0	42	2	0.00	1	1	
1	608	0	41	1	83807.86	1	0	
2	502	0	42	8	159660.80	3	1	
3	699	0	39	1	0.00	2	0	
4	850	0	43	2	125510.82	1	1	

5. Train/Test Split & Feature Scaling

```
In [10]: X = df.drop(columns=['Exited'])
y = df['Exited']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Train size:", X_train.shape, " Test size:", X_test.shape)

# Scale features (tree-based models don't strictly need it, but it keeps
# the pipeline consistent and helps the Logistic Regression meta-model in stack
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Train size: (8000, 11) Test size: (2000, 11)

```
In [11]: # Helper to evaluate each model consistently and collect results
results = {}

def evaluate_model(name, model, X_te, y_te):
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:, 1]

    acc = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred)
    rec = recall_score(y_te, y_pred)
    f1 = f1_score(y_te, y_pred)
    auc = roc_auc_score(y_te, y_proba)

    print(f"--- {name} ---")
    print(f"Accuracy: {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall: {rec:.4f}")
    print(f"F1 Score: {f1:.4f}")
    print(f"ROC-AUC: {auc:.4f}\n")
    print(classification_report(y_te, y_pred, target_names=['Stayed', 'Churned']))

    cm = confusion_matrix(y_te, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Stayed', 'Churned'])
```

```

disp.plot(cmap='Blues')
plt.title(f'{name} - Confusion Matrix')
plt.show()

results[name] = {'Accuracy': acc, 'Precision': prec, 'Recall': rec,
                 'F1 Score': f1, 'ROC-AUC': auc}

return y_proba

```

6. Model 1: Bagging

Bootstrap Aggregating — trains many decision trees in parallel, each on a random bootstrap sample of the training data, then combines them by majority vote. Reduces variance.

```

In [12]: bagging_model = BaggingClassifier(
            estimator=DecisionTreeClassifier(random_state=42),
            n_estimators=100,
            max_samples=0.8,          # each tree sees 80% of the rows (sampled with repla
            bootstrap=True,
            random_state=42,
            n_jobs=-1
        )
        bagging_model.fit(X_train_scaled, y_train)

        proba_bag = evaluate_model('Bagging', bagging_model, X_test_scaled, y_test)

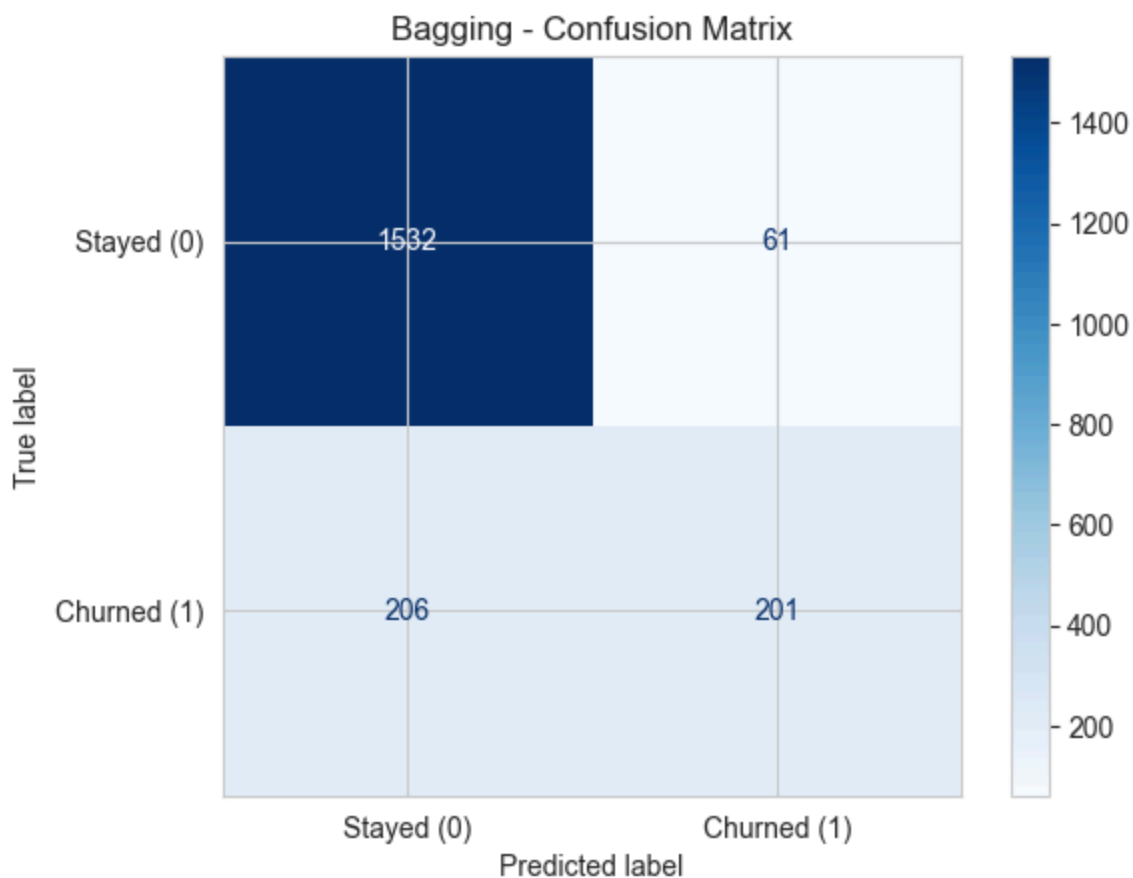
```

```

--- Bagging ---
Accuracy:  0.8665
Precision: 0.7672
Recall:    0.4939
F1 Score:  0.6009
ROC-AUC:   0.8407

```

	precision	recall	f1-score	support
Stayed	0.88	0.96	0.92	1593
Churned	0.77	0.49	0.60	407
accuracy			0.87	2000
macro avg	0.82	0.73	0.76	2000
weighted avg	0.86	0.87	0.85	2000



7. Model 2: AdaBoost

Adaptive Boosting — trains shallow trees sequentially; each round increases the weight of misclassified samples so the next learner focuses on the hard cases. Reduces bias.

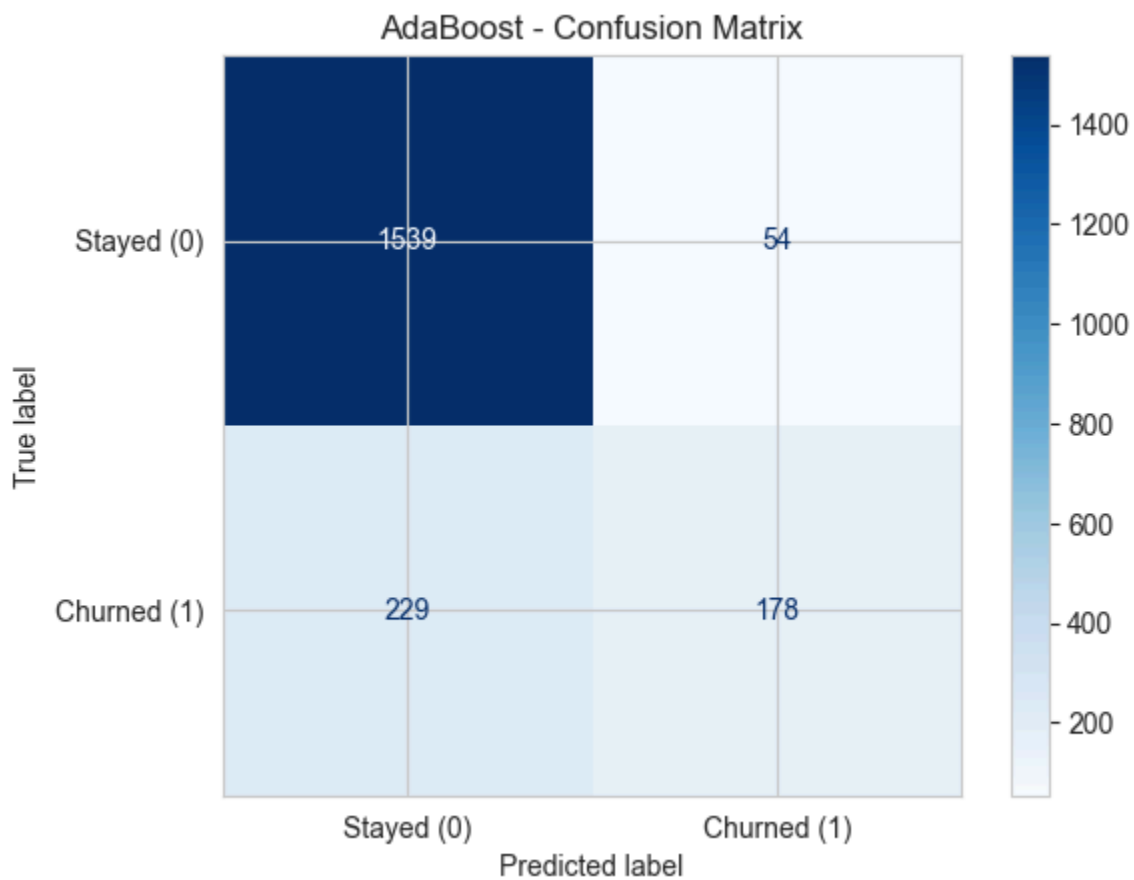
```
In [13]: adaboost_model = AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42), # decision  
    n_estimators=200,  
    learning_rate=0.5,  
    random_state=42  
)  
adaboost_model.fit(X_train_scaled, y_train)  
  
proba_ada = evaluate_model('AdaBoost', adaboost_model, X_test_scaled, y_test)
```

```

--- AdaBoost ---
Accuracy: 0.8585
Precision: 0.7672
Recall: 0.4373
F1 Score: 0.5571
ROC-AUC: 0.8511

```

	precision	recall	f1-score	support
Stayed	0.87	0.97	0.92	1593
Churned	0.77	0.44	0.56	407
accuracy			0.86	2000
macro avg	0.82	0.70	0.74	2000
weighted avg	0.85	0.86	0.84	2000



8. Model 3: XGBoost

Extreme Gradient Boosting — regularized gradient boosting where each new tree fits the residual errors of the ensemble so far. `scale_pos_weight` compensates for the ~4:1 class imbalance.

```
In [14]: # Ratio of negative to positive class to counter imbalance
ratio = (y_train == 0).sum() / (y_train == 1).sum()
print(f"scale_pos_weight = {ratio:.2f}")

xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=ratio,
    eval_metric='logloss',
    random_state=42
)
xgb_model.fit(X_train_scaled, y_train)

proba_xgb = evaluate_model('XGBoost', xgb_model, X_test_scaled, y_test)
```

scale_pos_weight = 3.91

--- XGBoost ---

Accuracy: 0.8075

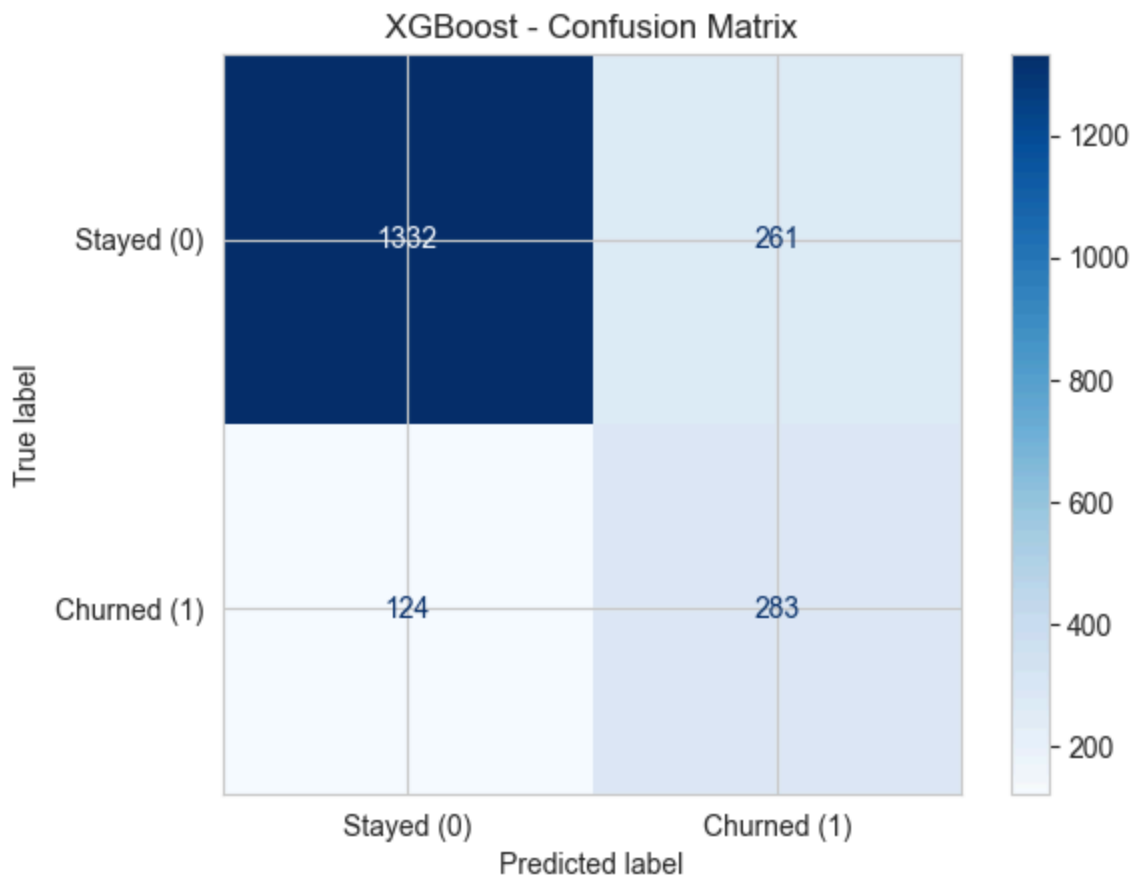
Precision: 0.5202

Recall: 0.6953

F1 Score: 0.5952

ROC-AUC: 0.8535

	precision	recall	f1-score	support
Stayed	0.91	0.84	0.87	1593
Churned	0.52	0.70	0.60	407
accuracy			0.81	2000
macro avg	0.72	0.77	0.73	2000
weighted avg	0.83	0.81	0.82	2000



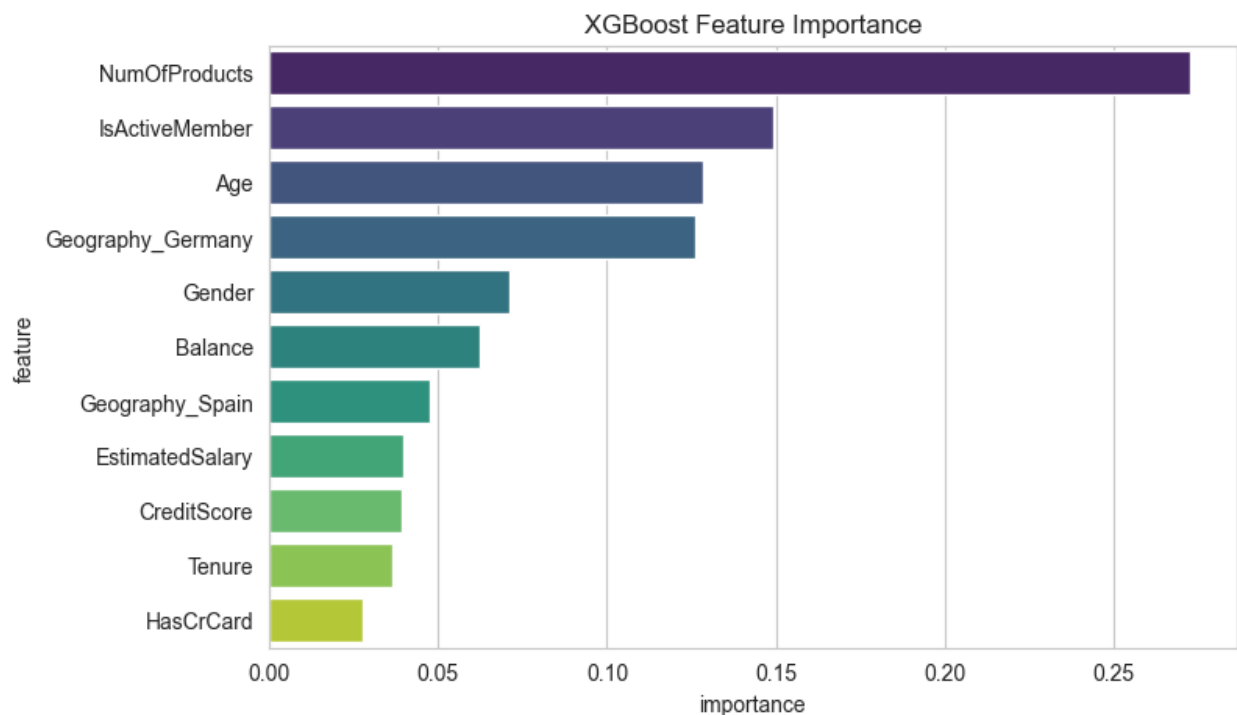
```
In [15]: # XGBoost feature importance
importance_df = pd.DataFrame({
    'feature': X.columns,
    'importance': xgb_model.feature_importances_
}).sort_values(by='importance', ascending=False)

plt.figure(figsize=(8, 5))
sns.barplot(x='importance', y='feature', data=importance_df, palette='viridis')
plt.title('XGBoost Feature Importance')
plt.show()
```

C:\Users\tusha\AppData\Local\Temp\ipykernel_13088\1613227009.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='importance', y='feature', data=importance_df, palette='viridis')
```



9. Model 4: Stacking

Stacking — trains diverse base models (Random Forest, XGBoost, KNN), then a Logistic Regression **meta-model** learns the best way to combine their predictions. Uses 5-fold cross-validated predictions internally to avoid leakage.

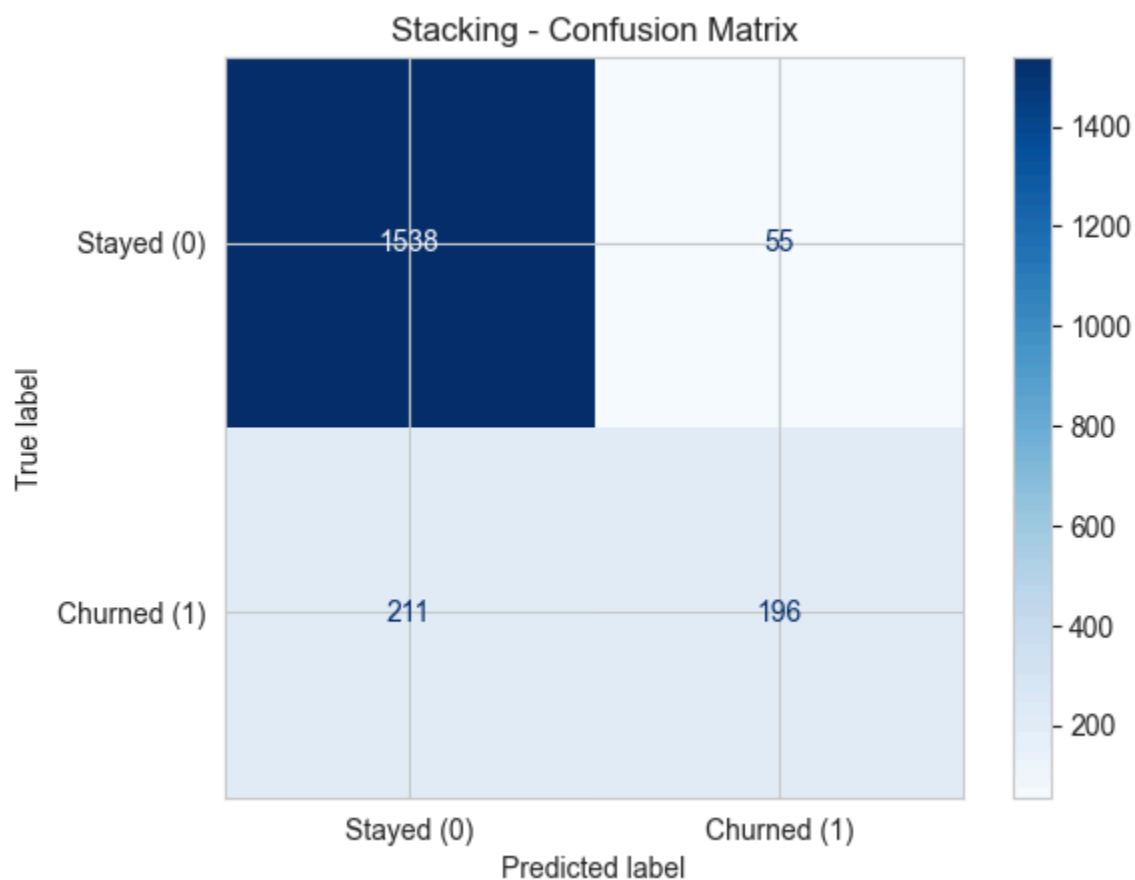
```
In [16]: base_learners = [  
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1,  
    ('xgb', XGBClassifier(n_estimators=200, learning_rate=0.1, max_depth=4,  
                           eval_metric='logloss', random_state=42)),  
    ('knn', KNeighborsClassifier(n_neighbors=15))  
]  
  
stacking_model = StackingClassifier(  
    estimators=base_learners,  
    final_estimator=LogisticRegression(max_iter=1000),  
    cv=5, # out-of-fold predictions train the meta-model (prevents  
    n_jobs=-1  
)  
stacking_model.fit(X_train_scaled, y_train)  
  
proba_stack = evaluate_model('Stacking', stacking_model, X_test_scaled, y_test)
```

```

--- Stacking ---
Accuracy: 0.8670
Precision: 0.7809
Recall: 0.4816
F1 Score: 0.5957
ROC-AUC: 0.8647

```

	precision	recall	f1-score	support
Stayed	0.88	0.97	0.92	1593
Churned	0.78	0.48	0.60	407
accuracy			0.87	2000
macro avg	0.83	0.72	0.76	2000
weighted avg	0.86	0.87	0.85	2000



10. Final Model Comparison

```

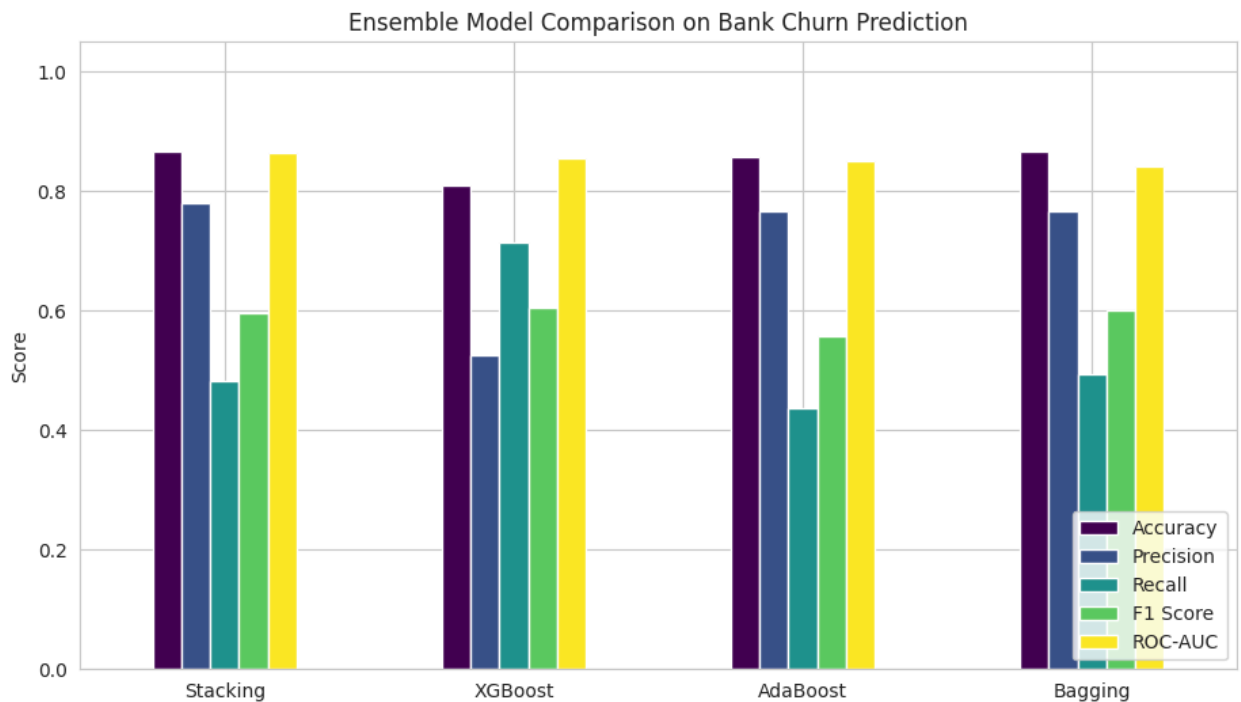
In [16]: comparison_df = pd.DataFrame(results).T.sort_values(by='ROC-AUC', ascending=False)
comparison_df.round(4)

```

Out[16]:

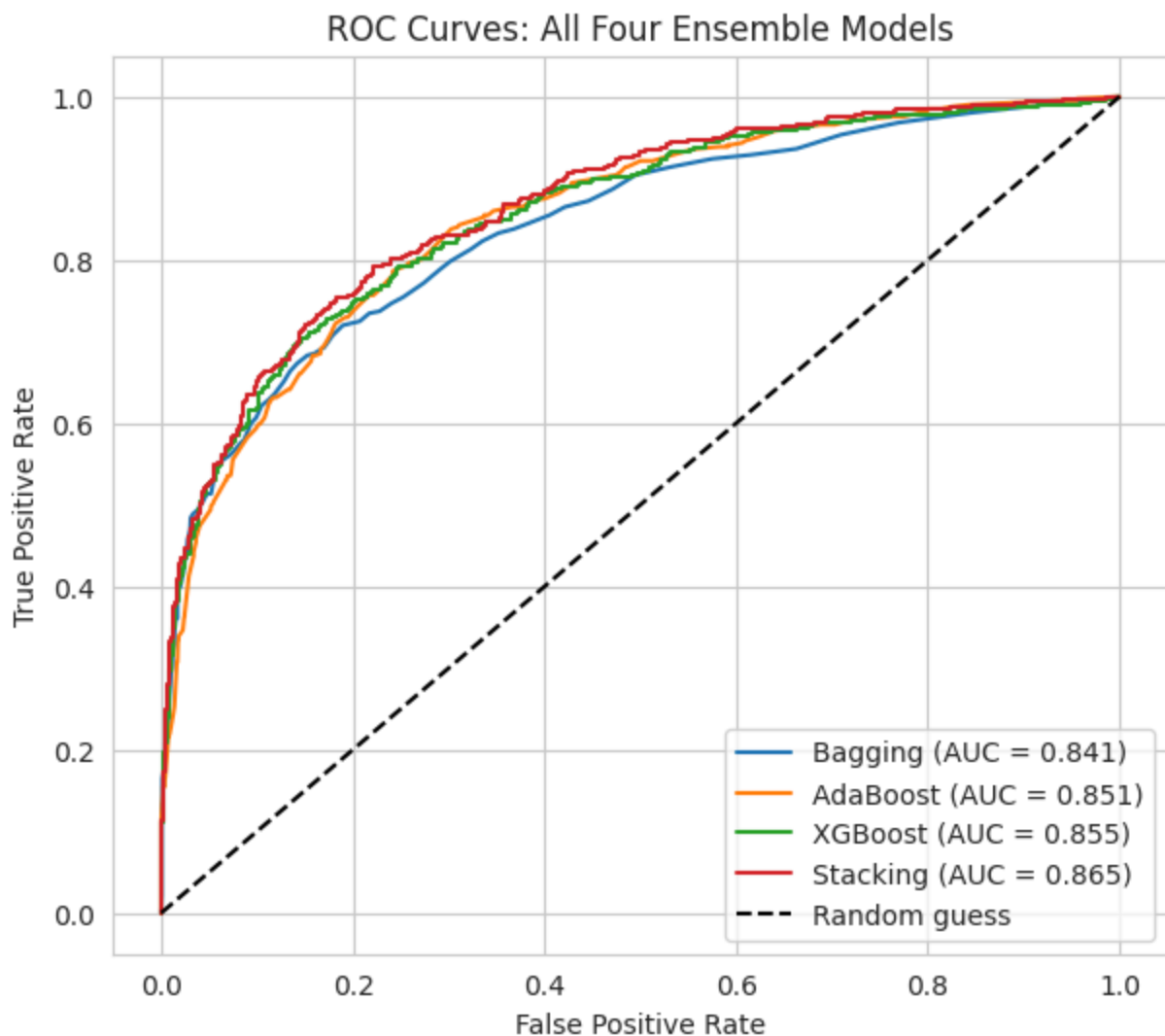
	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Stacking	0.8670	0.7809	0.4816	0.5957	0.8647
XGBoost	0.8105	0.5253	0.7150	0.6056	0.8553
AdaBoost	0.8585	0.7672	0.4373	0.5571	0.8511
Bagging	0.8665	0.7672	0.4939	0.6009	0.8407

```
In [17]: comparison_df.plot(kind='bar', figsize=(11, 6), colormap='viridis')
plt.title('Ensemble Model Comparison on Bank Churn Prediction')
plt.ylabel('Score')
plt.xticks(rotation=0)
plt.ylim(0, 1.05)
plt.legend(loc='lower right')
plt.show()
```



```
In [18]: # ROC curves for all four models on one plot
plt.figure(figsize=(7, 6))
for name, proba in [('Bagging', proba_bag), ('AdaBoost', proba_ada),
                    ('XGBoost', proba_xgb), ('Stacking', proba_stack)]:
    fpr, tpr, _ = roc_curve(y_test, proba)
    auc = roc_auc_score(y_test, proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves: All Four Ensemble Models')
plt.legend()
plt.show()
```



Summary

- Preprocessed the 10,000-customer bank churn dataset (dropped ID columns, encoded `Gender` and `Geography`).
- Applied four ensemble techniques on the same stratified train/test split:
 - **Bagging** — 100 decision trees on bootstrap samples (variance reduction)
 - **AdaBoost** — 200 decision stumps trained sequentially on re-weighted data (bias reduction)
 - **XGBoost** — 300 regularized boosted trees with `scale_pos_weight` to handle the ~20% churn imbalance
 - **Stacking** — Random Forest + XGBoost + KNN combined by a Logistic Regression meta-model
- Compared all models with accuracy, precision, recall, F1, ROC-AUC, confusion matrices, and overlaid ROC curves.

- Note: because only ~20% of customers churn, **ROC-AUC, recall, and F1 matter more than raw accuracy** here — a model that predicts "no churn" for everyone would already score ~80% accuracy while being useless.

Ideas to extend: hyperparameter tuning with `GridSearchCV` / `Optuna` , SMOTE oversampling for the minority class, threshold tuning to trade precision vs. recall, or SHAP values to explain individual churn predictions.